

=====

PROFESSIONAL AMR FLEET COMMAND CENTER
UI/UX + FRONTEND + LIVING DIGITAL-TWIN EXPERIENCE
SIH 2026 / SIH26123

=====

ROLE

=====

Act as a principal-level multidisciplinary product team working together:

- Principal Product Designer
- Senior UX Architect
- Senior Frontend Architect
- Senior Frontend Engineer
- Robotics Control-Interface Designer
- Industrial HMI Designer
- Digital-Twin Experience Designer
- Data Visualization Specialist
- Interaction / Motion Designer
- Performance Engineer
- Accessibility Specialist
- QA / Test Engineer
- Design-System Engineer
- Usability / Human-Factors Specialist

Your responsibility is to inspect the existing SWARMOS frontend and transform it into an exceptional, production-quality robotics fleet command-and-control experience.

Do not approach this as a basic dashboard redesign.

Do not approach this as a visual mockup.

Do not optimize for screenshots alone.

Build a coherent operational product.

The final product must feel credible as a serious robotics/logistics control platform that could be demonstrated to:

- robotics engineers
- warehouse operations teams
- fleet managers
- product leaders
- senior frontend engineers
- technical evaluators
- demanding SIH judges

The fact that this is being developed in a hackathon context does NOT justify reducing the quality bar.

The implementation should be ambitious in product quality, while remaining technically realistic and maintainable.

=====

NON-NEGOTIABLE QUALITY STANDARD

=====

The final result must be:

- professional
- operational
- technically credible
- visually distinctive
- precise
- calm
- intelligent
- trustworthy
- responsive
- performant
- accessible
- coherent
- genuinely functional

Do not settle for:

- generic SaaS dashboards
- generic AI dashboards
- dark cards on a black background
- cyberpunk interfaces
- gaming HUDs
- futuristic concept art
- glowing-card collections
- fake analytics
- fake simulation
- fake robot movement

- dead buttons
- incomplete workflows
- static warehouse graphics
- decorative UI that does not contribute to operation

The sophistication must come from:

- information architecture
- spatial visualization
- interaction quality
- system transparency
- meaningful motion
- operational workflows
- data integrity
- visual hierarchy
- performance
- consistency
- craftsmanship

NOT from superficial visual effects.

THE CENTRAL PRODUCT IDEA

SWARMOS is not:

"a dashboard with robots on a map."

SWARMOS is:

"A living digital twin and fleet operations control system where autonomous mobile robots actually perform work, coordinate with one another, encounter operational conditions, adapt to changes, recover from failures, and generate measurable operational data."

The core product loop is:

```

REAL WAREHOUSE
  ↓
REAL FLEET
  ↓
REAL TASKS
  ↓
REAL ROBOT MOVEMENT
  ↓
REAL TRAFFIC
  ↓
REAL CONGESTION
  ↓
REAL CONFLICT
  ↓
REAL DISTRIBUTED DECISION
  ↓
REAL ACTION
  ↓
REAL REROUTING / WAITING / YIELDING
  ↓
REAL BLOCKAGE / FAILURE
  ↓
REAL TASK REASSIGNMENT
  ↓
REAL RECOVERY
  ↓
REAL EVENTS
  ↓
REAL METRICS
  ↓
REAL BENCHMARKS
  ↓
REAL ANALYTICS

```

This chain must be reflected throughout the interface.

Do not break this relationship by creating a decorative UI layer that is disconnected from system behavior.

PRIMARY SCOPE

This task is primarily about:

- UI
- UX
- frontend

- interaction design
- digital twin
- simulation presentation
- map visualization
- real-time visualization
- motion
- data visualization
- accessibility
- performance
- frontend reliability
- usability
- information architecture
- design systems

DO NOT redesign or implement:

- backend architecture
- backend services
- database architecture
- APIs
- server infrastructure
- robotics algorithms
- backend communication architecture
- backend simulation architecture

Assume those are managed by another team member.

You may create frontend adapters, state handling, visualization logic, UI orchestration, client-side rendering, and presentation-layer logic needed to integrate correctly with the existing system.

Do not turn this assignment into a backend rewrite.

=====

SOURCE MATERIAL – TREAT AS AUTHORITATIVE

=====

Use all available SWARMOS source material supplied with this task:

1. Complete SWARMOS UI/UX research report.
2. Complete 14-page SWARMOS UI/UX Workflow Blueprint PDF.
3. Existing SWARMOS Master Execution Prompt.
4. Current SWARMOS UI screenshots.
5. Existing architecture diagrams.
6. Existing project/folder structure.
7. Existing frontend implementation.
8. Supplied web-dashboard reference.
9. Existing SWARMOS problem/domain context.

Do not ignore useful information from these sources.

The 14-page UI/UX Blueprint is particularly important because it establishes:

- problem understanding
- users
- feature inventory
- operator workflows
- information architecture
- wireframe direction
- design system
- high-fidelity UI
- hero live-map concept
- robot details
- Edge-AI visibility
- conflict-resolution visualization
- purposeful animation
- frontend implementation
- realtime connection
- AI reasoning
- analytics
- simulation
- benchmark/demo workflow

However, DO NOT simply reproduce the PDF.

Use it as the functional and UX foundation.

Improve it significantly using the complete SWARMOS research.

The supplied web-dashboard reference is visual inspiration only.
Do not copy its exact appearance.

The supplied SWARMOS architecture is the source of actual operational concepts.

Do not invent unrelated product capabilities.

=====

FIRST PHASE – INSPECT BEFORE CHANGING

=====

Before implementing substantial changes:

inspect the repository thoroughly.

Understand:

- current framework
- routing
- folder structure
- components
- pages
- styles
- design system
- state management
- API usage
- realtime behavior
- WebSocket usage
- simulation integration
- robot rendering
- task system
- incident system
- analytics
- benchmark system
- replay
- current controls
- current interactions
- current data contracts

Audit every current screen.

For each screen identify:

- what works
- what partially works
- what is broken
- what is fake
- what is duplicated
- what is visually weak
- what is confusing
- what is unnecessary
- what is valuable

Internally classify features:

KEEP
IMPROVE
REPAIR
RESTRUCTURE
REPLACE ONLY WHEN NECESSARY

Do not blindly rebuild the application.

Do not destroy useful functionality merely because its current presentation is poor.

Do not delete a useful feature to hide an implementation problem.

Repair it.

=====

NO-FAKE-FUNCTIONALITY POLICY

=====

Never create:

- dead buttons
- fake buttons
- fake toggles
- fake filters
- fake charts
- fake metrics
- fake incidents
- fake telemetry
- fake benchmark values
- fake AI explanations
- fake robot movement
- fake simulation states
- fake system health

- fake realtime status
- decorative controls with no behavior
- navigation destinations that do nothing

If a feature exists in the product and is useful:

make it work.

If data is genuinely unavailable:

show:

- Unavailable
- No Data
- Not Measured
- Not Connected
- Waiting for Data

Do not invent believable values.

=====

DATA INTEGRITY – ABSOLUTE

=====

Every visible metric, graph, status and count must have a real source.

For EVERY KPI:

Ask:

"What actual data produces this number?"

For EVERY CHART:

Ask:

"Where does every plotted data point come from?"

For EVERY STATUS:

Ask:

"What actual state produces this status?"

For EVERY INCIDENT:

Ask:

"What real event produces this incident?"

For EVERY BENCHMARK:

Ask:

"What actual run produced this result?"

For EVERY AI EXPLANATION:

Ask:

"What actual decision/context produced this explanation?"

If there is no legitimate source:

do not fabricate it.

Either:

- connect it correctly
- label it unavailable
- remove it

Never use hard-coded "impressive" values for presentation.

=====

REAL SIMULATION IS THE CORE EXPERIENCE

=====

The frontend must visualize a living simulation.

50 AMRs is the primary demonstration target.

100+ AMRs is a stress target where technically practical.

The robots must actually operate.

They should have meaningful work.

Example:

R01 → Pickup → Dropoff
R02 → Pickup → Dropoff
R03 → Charging
R04 → Waiting for right-of-way
R05 → Delivering
R06 → Rerouting
R07 → Congested
...
R27 → Fault
R28 → Task reassignment from R27

Do not have robots randomly wander around.

Do not create decorative loops.

Do not make 50 identical moving markers.

The fleet should look like a real operating system.

=====

REAL ROBOT WORK CYCLE

=====

Where supported by the current simulation:

IDLE
→ ASSIGNED
→ MOVING TO PICKUP
→ PICKUP
→ MOVING TO DROPOFF
→ DROPOFF
→ COMPLETE
→ NEW TASK
→ REPEAT

Other legitimate states:

- WAITING
- YIELDING
- REROUTING
- CHARGING
- BLOCKED
- FAULT
- RECOVERING
- COMMUNICATION DEGRADED

Visual state must correspond to actual state.

=====

ROBOT MOVEMENT – NON-NEGOTIABLE

=====

Robots must move visibly and continuously.

Do NOT teleport.

Do NOT simply update their position as discrete jumps.

Do NOT animate an unrelated looping trajectory.

Use interpolation between real state updates.

The visual render loop should be independent from slower application state updates where appropriate.

Robot movement should communicate:

- position
- heading
- orientation
- velocity
- acceleration/deceleration where appropriate
- turning
- stopping
- route progression
- route change

Make the motion feel physically coherent.

When a robot turns:

it should not instantly rotate 90 degrees without context.

When a robot stops:

it should visually stop.

When a robot reroutes:

the route should visibly change.

When a robot receives a new task:

the destination and route should update.

When a robot fails:

the robot state should change.

When another robot receives its task:

that robot should actually begin the relevant task flow.

The user should be able to watch the warehouse for several seconds and clearly understand:

"These robots are actually working."

=====

DIGITAL TWIN – VISUAL HERO

=====

The Live Warehouse is the most important visual surface in SWARMOS.

The warehouse itself is the environment.

Do NOT put a random animated background behind a weak warehouse map.

The digital twin must contain believable:

- storage racks
- shelves
- aisle corridors
- intersections
- pickup locations
- dropoff locations
- staging areas
- charging stations
- docks
- restricted zones
- obstacles
- blocked areas
- warehouse landmarks
- zone labels

The environment should communicate spatial relationships.

Operators should immediately understand:

WHERE the robots are
WHERE tasks are
WHERE traffic is
WHERE conflicts happen
WHERE blockages occur
WHERE chargers are
WHERE important zones are

=====

BACKGROUND – STRICT REQUIREMENT

=====

Replace any generic background direction.

DO NOT use:

- AI-generated futuristic wallpaper
- futuristic cities
- generic sci-fi art
- hacker grids
- cyberpunk scenery
- random robot photographs
- decorative particles
- unrelated abstract animations

- noisy animated backgrounds
- giant glowing grids

Preferred approach:

THE WAREHOUSE ITSELF IS THE VISUAL ENVIRONMENT.

Use:

- sophisticated warehouse geometry
- refined floor treatment
- realistic aisle relationships
- subtle technical spatial structure
- subdued environment layers
- restrained digital-twin styling
- meaningful state-driven visual changes

The environment may have subtle depth.

It must remain subordinate to operational information.

Do not make the background compete with the robots, incidents or controls.

=====

2D / 2.5D / 3D

=====

Use professional judgment.

Primary operational mode:

2D or carefully designed 2.5D top-down warehouse visualization.

Optional secondary mode:

3D inspection/presentation view where it adds genuine value.

Do NOT use 3D simply because it looks impressive.

3D must never make:

- routes harder to understand
- conflict relationships harder to see
- robot selection harder
- performance worse
- labels unreadable

A beautiful 2D digital twin is preferable to a slow, confusing 3D world.

=====

AMR VISUALIZATION

=====

The current problem of tiny button-like robots must be eliminated.

Do not render robots as:

- dots
- generic circles
- tiny control buttons
- meaningless colored points

Create a recognizable professional AMR visual primitive.

Each AMR must communicate:

- body/chassis
- orientation
- direction
- status
- selection
- task relationship

At zoomed-out levels:

show a simplified but recognizable silhouette.

At zoomed-in levels:

reveal:

- robot ID
- route

- task
- destination
- nearby relationships
- state

Do not use game-character styling.

Do not make robots cartoonish.

Do not use unnecessarily detailed 3D models.

=====

50-100 ROBOT VISUALIZATION

=====

50 robots must remain understandable simultaneously.

Do not permanently display large labels over every robot.

Use scale-aware detail:

ZOOM OUT:

- body
- heading
- state

ZOOM IN:

- ID
- route
- task
- telemetry context

Use focus/selection behavior:

selected robot becomes visually dominant

nearby relevant robots remain visible

unrelated robots become slightly subdued if necessary

Use clustering or detail reduction only when required by density.

Do not hide meaningful operational state.

=====

RENDERING STRATEGY

=====

Evaluate the current implementation and choose the most appropriate rendering approach.

Possible technologies:

- Canvas
- WebGL
- PIXIJS
- Three.js
- React Three Fiber
- SVG
- hybrid Canvas + HTML/SVG

Do not choose based on popularity.

Choose based on:

- 50+ moving AMRs
- map overlays
- route rendering
- performance
- interaction
- accessibility
- maintainability

Strong default direction:

dynamic map/robot layer:

Canvas/WebGL/Pixi-like renderer where useful

UI/semantic layer:

React/HTML/SVG

Use accessible overlays for:

- selected entities
- labels

- tooltips
- inspector
- critical information

RENDER LOOP / REACT STATE

Do not let high-frequency robot movement force the entire React tree to rerender.

Separate:

HIGH-FREQUENCY VISUAL STATE
from
LOW-FREQUENCY OPERATIONAL UI STATE.

Use where justified:

- requestAnimationFrame
- interpolation
- selective subscriptions
- batching
- memoization
- normalized state
- virtualization
- Web Workers where genuinely useful

Profile before optimizing.

Do not sacrifice usability simply for maximum FPS.

ROUTE VISUALIZATION

Routes should be meaningful.

Represent:

- current route
- future route
- destination
- route progress
- completed route history where useful
- route ownership
- alternative route where relevant

When rerouting:

OLD ROUTE
→ de-emphasizes/fades

NEW ROUTE
→ appears clearly

ROBOT
→ transitions smoothly onto the new route

This must tell the operator:

"SWARMOS changed the plan."

CONFLICT DETECTION UX

Do not show only:

"Conflict detected."

Show the actual spatial story.

For an actual conflict event:

- involved robots
- their directions
- conflict point
- affected routes
- severity
- time-to-conflict where available
- priority
- chosen response

- final outcome

Use:

DETECTED
→ WHY
→ DECISION
→ ACTION
→ RESOLVED

Example:

R27 → Junction J4
R34 → Junction J4

Potential conflict

↓

R34 higher task priority

↓

R27 yields

↓

R34 continues

↓

R27 resumes

↓

Conflict resolved

The visual simulation must actually reflect the decision.

=====

DEADLOCK UX

=====

For real or intentionally induced deadlocks:

show:

- affected robots
- affected region
- blocked dependencies
- why movement is blocked
- recovery decision
- route changes
- recovery state

Where useful, show the relationship as a dependency chain/cycle.

Example:

R07 → waits for R18
R18 → waits for R22
R22 → waits for R07

DEADLOCK

Then:

SWARMOS identifies cycle

→ changes priority / route

→ breaks cycle

→ robots resume

The map and incident view should tell the same story.

=====

CONGESTION UX

=====

Congestion should be a spatial condition, not merely a KPI.

Visualize:

- traffic density
- waiting
- route density
- affected robots
- affected tasks
- bottleneck location
- severity

Provide an optional:

Congestion Heatmap

The intensity must come from actual data.

Allow users to turn layers on/off.

Do not permanently cover the map with color.

=====

BLOCKED AISLE UX

=====

A blockage must have real downstream consequences.

When an aisle becomes blocked:

1. warehouse state changes
2. blocked area becomes visible
3. affected robots are identified
4. affected tasks are identified
5. affected routes are identified
6. rerouting occurs where supported
7. incident appears
8. recovery occurs
9. metrics change

Do not simply add a red line and leave all robots behaving as though nothing happened.

=====

ROBOT FAILURE UX

=====

Failure must produce a real operational sequence.

Example:

R27 fails

↓

R27 = FAULT

↓

current mission becomes AT RISK

↓

affected route is reconsidered

↓

replacement robot selected

↓

task reassigned

↓

replacement robot starts work

↓

fleet recovers

Show this visually across:

- map
- robot inspector
- task inspector
- incident center
- event timeline
- metrics

MISSION / TASK MANAGEMENT

Tasks are core operational objects.

Provide:

- task list/table
- filtering
- sorting
- search
- status
- priority
- zone
- assigned robot
- ETA
- wait time
- SLA risk
- reassignment
- reprioritization
- cancellation
- detail inspection

Use virtualization for large lists.

MISSION COMPOSER

Create a professional mission/task creation workflow.

Example:

NEW MISSION

Pickup
[P-07]

Destination
[D-14]

Priority
[Normal / High / Critical]

Payload
[18 kg]

Capability
[Standard AMR]

Optional:
preferred zone

Then:

SYSTEM RECOMMENDATION

R27

Why?

- ✓ closer
- ✓ sufficient battery
- ✓ capacity available
- ✓ compatible route
- ✓ lower congestion
- ✓ currently available

Then:

[REVIEW & DISPATCH]

If the backend exposes recommendation details, show them.

If not, do not fabricate AI reasoning.

TASK DETAIL

Selecting a mission should reveal:

- task ID

- pickup
- destination
- priority
- assigned robot
- status
- ETA
- wait time
- SLA
- route
- event timeline
- reassignment history
- related incidents

Clicking the assigned robot should preserve context and open the robot inspector/focus.

```
=====
ROBOT INSPECTOR
=====
```

Selecting a robot should open a right-side contextual inspector.

Sections:

IDENTITY

- Robot ID
- status
- mode

HEALTH

- battery
- communication
- available health data

MISSION

- task ID
- pickup
- dropoff
- priority
- ETA

LOCATION

- position
- heading
- speed
- next waypoint

ROUTE

- current route
- progress
- destination

NEARBY ROBOTS

- relevant nearby units
- conflicts if present

EVENTS

- assignments
- waits
- reroutes
- completion
- faults
- recovery

DECISION TRACE

- what happened
- why
- alternatives if available
- decision
- effect

ACTIONS

- only actions actually supported

Prefer a drawer over leaving the map context.

```
=====
DECISION INSPECTOR
=====
```

This is a major SWARMOS differentiator.

Make autonomous intelligence understandable.

When a system decision occurs, show:

DECISION
WHY
CONTEXT
ALTERNATIVES
SELECTED ACTION
IMPACT
OUTCOME

Example:

R27 REROUTED

Reason:
Aisle 7 blocked

Context:
Task priority = High
Battery = 82%
Alternate path = Available
Conflict count = 1

Selected:
Route A → C → F

Measured/estimated impact:
+3.8 seconds

Outcome:
Robot resumed transit

Only show fields supported by real system data.

=====

EDGE / DISTRIBUTED INTELLIGENCE

=====

The Blueprint explicitly requires the distributed intelligence to be visible.

Do not merely show:

"AI Enabled."

Where actual system information exists, show:

- edge node state
- connected robots
- inference latency
- communication state
- recent decisions
- decision events

More importantly, show the operational meaning.

Example:

EDGE NODE 03
ONLINE

6 robots connected

Recent decision:
R27 yield

Reason:
R34 priority > R27

Decision latency:
8 ms

If these numbers are not real:

do not fabricate them.

=====

FLEET OVERVIEW

=====

Fleet Overview must answer:

"How is the fleet operating right now?"

Above the fold:

4–6 high-value metrics.

Possible:

- total robots
- active
- idle
- charging
- faulted
- active tasks
- completed tasks
- throughput
- incidents
- congestion

Do not create 20 equal cards.

Prioritize exceptions.

Use trends only when they answer useful questions.

Include:

- fleet health
- top active incidents
- compact activity timeline
- map preview if useful

A supervisor should understand fleet state within seconds.

=====

INCIDENT CENTER

=====

Use exception-first design.

Possible incidents:

- collision risk
- deadlock
- blocked aisle
- robot failure
- low battery
- communication degradation
- task SLA risk
- congestion spike

Every incident should tell:

WHAT?
WHERE?
WHEN?
WHO?
WHICH TASKS?
SEVERITY?
WHAT DID SWARMOS DO?
WHAT CAN THE OPERATOR DO?
WHAT HAPPENED AFTERWARD?

Possible actions:

- Locate
- Inspect
- Reroute
- Reassign
- Acknowledge
- Resolve
- Escalate

Only expose actions that the underlying system can actually perform.

=====

INCIDENT GROUPING

=====

Prevent alarm fatigue.

If multiple related alerts originate from the same area:

group them.

Example:

"7 robots affected by congestion in Zone C"

Then allow drill-down.

Do not generate seven visually dominant warnings for the same operational condition.

=====

ALERT SYSTEM

=====

Use semantic status:

GREEN = healthy
AMBER = warning
RED = critical
BLUE = information

Never use color alone.

Every important alert must include:

- icon
- text
- severity
- context

Critical incidents should remain visible until addressed or explicitly resolved.

Do not use aggressive flashing.

Use controlled motion only when attention is required.

=====

MISSION TIMELINE

=====

Where task history exists, show a clean timeline:

Created
↓
Assigned

↓
Route Generated
↓
Started
↓
Pickup
↓
Transit
↓
Conflict / Reroute
↓
Dropoff
↓
Completed

Use actual timestamps/events.

This is especially useful in:

- task inspector
- replay
- incident investigation

=====

EVENT TIMELINE

=====

Provide an operational event stream where useful.

Examples:

12:31:04
R27 assigned T184

12:31:09
R27 entered J4

12:31:10
Conflict detected

12:31:10
R27 yielded

12:31:12
Conflict resolved
12:31:33

T184 completed

Use actual events.

Do not generate fake narrative events.

REPLAY / TIME TRAVEL

Replay should behave like a professional simulation-analysis tool.

Provide:

- play
- pause
- speed
- timeline scrubber
- step forward
- step backward
- event markers
- incident markers
- robot focus
- task focus

When the user changes time:

robot positions
task states
routes
incidents
decisions

must reflect that historical moment.

Do NOT simply animate current state backward.

REPLAY ↔ EVENT CORRELATION

Make events clickable.

Click:

"Conflict detected"

- replay jumps to that moment
- involved robots focus
- conflict point highlights
- incident/decision information appears

This makes historical investigation powerful.

SIMULATION LAB

Make the Simulation Lab one of the strongest areas of SWARMOS.

It should feel like a professional robotics "what-if laboratory."

NOT:

- game settings
- collection of sliders
- basic form
- fake demo screen

Provide where supported:

- fleet size
- task count/load
- task injection rate
- blocked aisles
- obstacles
- robot failures
- communication degradation
- battery stress
- task-priority mix

- simulation duration
- simulation speed

SCENARIO PRESETS

Useful presets:

- Normal Operations
- Rush Hour
- High Traffic
- Intersection Conflict
- Aisle Blockage
- Robot Failure
- Deadlock Challenge
- Battery Stress
- Communication Degradation
- 50-Robot Stress Test
- Grand Finale Showcase

Each preset MUST configure the actual simulation state.

Do not change only labels.

SCENARIO BUILDER

Where supported, allow spatial editing.

Examples:

Select aisle
→ mark blocked

Select robot
→ inject failure

Select zone
→ increase task load

Select region
→ apply congestion

Configure:

fleet size
task intensity
failure conditions
communication quality
battery stress

Provide scenario preview.

The preview must correspond to the actual configuration.

SIMULATION CONTROLS

Controls must work:

START
PAUSE
RESUME
RESET
RESTART
SPEED
STEP where supported

Pause means actual pause.

Resume means actual resume.

Speed means actual simulation-speed change.

Reset means actual state reset.

WOW SHOWCASE

Build a coherent real operational showcase.

Preferred sequence:

1. Start 50 AMRs.
2. AMRs continuously perform meaningful tasks.
3. Tasks complete.
4. New tasks appear.
5. Traffic increases.
6. Congestion develops.
7. Potential conflict occurs.
8. SWARMOS detects it.
9. Decision becomes visible.
10. Robot yields/waits/reroutes.
11. Conflict resolves.
12. Aisle blockage occurs.
13. Affected routes change.
14. A robot fails.
15. Its task becomes at risk.
16. A new robot is selected.
17. Task is reassigned.
18. Replacement robot moves.
19. Fleet recovers.
20. Tasks continue.
21. Simulation produces actual measurements.
22. Benchmark results become available.
23. Actual charts show the result.

The WOW moment is NOT:

"look at our animation."

The WOW moment is:

"the system encountered a difficult operational condition and autonomously handled it while the operator could see exactly what happened."

=====

WOW SHOWCASE MUST BE REAL

=====

Do not build a cinematic fake demo.

The showcase must use actual available simulation controls/state.

If a scenario can be triggered:

trigger it.

If a simulation event is generated:

visualize it.

If a metric changes:

show the actual change.

If the underlying system cannot support a behavior:

do not pretend that it does.

=====

BENCHMARKS

=====

Benchmarking is the proof layer.

Compare actual runs.

Potential metrics:

- total completion time
- average completion time
- average wait time
- P95 wait
- throughput
- collisions
- deadlocks
- reroutes
- recovery time
- path efficiency
- utilization
- coordination latency where available
- battery-related metrics where available

Support:

- scenario
- fleet size
- run
- configuration

Where multiple runs exist:

show:

- mean
- median where meaningful
- P95 where meaningful
- variability
- min/max where useful

Never hide variability to make results look better.

SIH TARGETS

Where the actual system measures them, support evaluation against:

- zero inter-robot collisions
- at least 20% reduction in total task completion time versus the defined baseline

DO NOT claim these were achieved unless actual benchmark data proves them.

The UI may show:

TARGET

ACTUAL

STATUS

If target is not met:

show actual performance.

Do not manufacture a PASS.

ANALYTICS

Analytics must answer operational questions.

Examples:

Which aisle creates the most congestion?

Which intersections generate the most conflicts?

Which robots have the most idle time?

How does throughput change with fleet size?

How does rerouting affect completion time?

How often do deadlocks occur?

Which tasks wait longest?

How quickly does the fleet recover?

What happens when fleet size rises?

Use appropriate charts only.

CHART TYPES

Prefer:

LINE CHART
for time-series trends

BAR CHART
for scenario comparison

HEATMAP

for spatial congestion/conflict density

DISTRIBUTION

for variability in completion/wait times

SCALING CURVE

for fleet-size analysis

TIMELINE

for events/incidents

SPARKLINE

only for compact genuine trends

Do NOT use charts because a page looks empty without them.

NO FAKE GRAPHICS

Absolutely do not use:

- static chart images
- decorative SVG charts
- screenshot graphs
- hard-coded line points
- random percentage values
- fake historical trends
- fake benchmark improvements

Use real chart components connected to actual data.

Every chart needs:

- title
- labels
- units
- correct scale
- tooltip
- meaningful legend where needed

BOTTLENECK ANALYSIS

Provide a useful operational bottleneck view.

Potential information:

- bottleneck location
- traffic density
- average wait
- queue size
- conflicts
- affected robots
- affected tasks
- delay contribution
- frequency

Link bottlenecks to the map.

Clicking a bottleneck should focus the map.

FLEET SCALING

Support actual experiment comparison for:

10
25
50
75
100+

where practical.

Visualize measured effects on:

- throughput
- completion time
- wait time
- conflicts
- coordination latency

Do not claim scalability without testing.

COMMAND PALETTE

Implement a real global command palette.

Shortcut:

Ctrl/Cmd + K

Possible commands:

Find Robot R27
Find Task T145
Open Live Map
Open Fleet
Open Incidents
Open Simulation Lab
Open Replay
Open Analytics
Open Benchmarks
Pause Simulation
Resume Simulation
Reset Simulation
Fit Fleet
Focus Robot
Show Congestion
Show Conflicts

Show Charging Robots

Every command must perform a real action.

GLOBAL SEARCH

Search should work across available entities:

- robots
- tasks
- incidents
- scenarios
- locations

Example:

search "R27"

→ result

→ click

→ map centers

→ robot selected

→ inspector opens

FOCUS MODE

Add a contextual focus mode where appropriate.

Example:

Select R27.

Then:

- R27 remains prominent
- route visible
- relevant nearby robots visible
- related task visible
- relevant conflict visible
- unrelated map elements become visually quieter

Do not hide important operational context.

MAP LAYERS

=====

Provide functional toggles for:

- Robots
- Routes
- Trails
- Tasks
- Incidents
- Congestion
- Conflicts
- Reservations
- Battery Risk
- Navigation Graph

Each layer must actually change the map.

Do not create cosmetic toggles.

=====

MAP CONTROLS

=====

Support:

- pan
- zoom
- fit fleet
- center selected robot
- focus incident
- focus task
- layer controls
- reset view

Keyboard support where practical.

=====

MAP INTERACTION

=====

Click:

robot
→ select

task
→ task context

incident
→ incident context

conflict
→ conflict explanation

location
→ map focus

Hover:

show concise context.

Avoid excessive permanent labels.

=====

ROLE-AWARE UX

=====

Where the system supports roles, consider:

SHIFT SUPERVISOR

- incidents
- live fleet
- tasks
- immediate actions

WAREHOUSE MANAGER

- throughput
- utilization
- benchmarks
- historical performance

MAINTENANCE

- faults
- battery

- robot health

SYSTEM / ENGINEERING

- realtime connection
- decision latency
- engine health
- events

Do not add role complexity if the system does not support it.

=====

VISUAL DESIGN DIRECTION

=====

Target feeling:

Futuristic
+
Premium
+
Industrial
+
Technical
+
Precise
+
Calm
+

Trustworthy
+
Operational

But NOT:

cyberpunk
gaming
hacker
sci-fi concept art
AI-template aesthetic

The UI should feel advanced because it is well engineered.

=====

DARK MODE

=====

Use:

deep charcoal
graphite
slate
near-neutral dark surfaces

Avoid pure black as the dominant background if possible.

Use off-white text rather than excessive pure white.

Use subtle borders.

Use restrained depth.

=====

COLOR SYSTEM

=====

Use a restrained semantic palette.

PRIMARY:
one controlled SWARMOS accent

SUCCESS:
green

WARNING:
amber

CRITICAL:
red

INFORMATION:
blue

Do not make every section colorful.

Reserve strong colors for meaningful states.

SURFACE SYSTEM

Prefer:

solid surfaces
subtle borders
light depth
modest radius

Do not use:

- glassmorphism everywhere
- giant blurred panels
- glowing cards
- giant shadows
- excessive rounded containers

TYPOGRAPHY

Use a professional highly legible sans-serif.

Good characteristics:

- readable at 1080p
- strong number rendering
- tabular numerals
- clear hierarchy

Possible font families include:

Inter
Geist
Manrope
or equivalent professional UI font

Use tabular numerals for:

- KPIs
- metrics
- IDs
- timestamps
- latency
- tables

Do not use sci-fi fonts.

Do not make critical information tiny.

SPACING

Use a coherent spacing scale.

Prefer a 4/8-based rhythm.

Maintain:

- consistent padding
- consistent gutters
- consistent section spacing

Do not randomly change spacing between screens.

ICONOGRAPHY

Use one coherent icon system.

Icons should have clear meaning.

Do not use random emojis in the professional control center UI.

Do not use decorative icon collections.

NAVIGATION

Use a stable left sidebar.

Recommended structure:

OPERATE

- Overview
- Live Warehouse
- Fleet
- Missions
- Incidents

SIMULATE

- Simulation Lab

ANALYZE

- Replay
- Analytics
- Bottlenecks
- Benchmarks

SYSTEM

- Edge / Decisions
- System Health
- Settings

Adapt this if the existing implementation has a stronger structure.

Do not duplicate screens unnecessarily.

TOP BAR

The top operational bar should communicate:

- SWARMOS identity
- system state
- simulation state
- simulation time
- realtime connection
- search
- command palette
- global controls

Do not overload it.

THREE-PANE WORKSPACE

Preferred conceptual model:

LEFT

Navigation

CENTER

Primary workspace

RIGHT

Context / Inspector

For the Live Warehouse:

CENTER = DIGITAL TWIN

RIGHT = contextual inspector

Do not permanently occupy a large portion of the screen with generic metrics.

OVERVIEW SCREEN

Purpose:

fleet health in seconds.

Primary question:

"How is the fleet operating?"

Above the fold:

fleet health
4-6 meaningful KPIs
urgent incidents
compact activity context

Optional:

small map preview

Do not create a wall of equal cards.

=====

LIVE WAREHOUSE SCREEN

=====

This is the hero screen.

The judge should immediately understand:

there is a warehouse
there are many robots
they are doing work
they are moving
they have tasks
traffic exists
coordination exists
the system responds to events

The map should occupy the majority of the workspace.

=====

FLEET SCREEN

=====

Provide an operational view of all robots.

Possible columns:

Robot ID
Status
Battery
Task
Zone
ETA
Connection
Alerts

Allow:

search

filter
sort
status filtering
zone filtering
battery filtering

Click robot:

focus map + inspector.

=====

INCIDENT SCREEN

=====

Exception first.

The user should immediately know:

what needs attention now?

Group related alerts.

Support drill-down.

=====

EDGE / DECISION SCREEN

=====

If actual edge-node information exists:

show:

node
status
robots connected
inference/processing data
recent decision events

But focus more on decisions and operational consequences than raw infrastructure numbers.

=====

SIMULATION SCREEN

=====

Make it visually and functionally exceptional.

Possible layout:

LEFT:
scenario configuration

CENTER:
live warehouse

RIGHT:
live metrics/events

BOTTOM:
simulation timeline

The user should be able to watch the scenario unfold.

=====

BENCHMARK SCREEN

=====

Make actual evidence visually dominant.

Top:

scenario
fleet size
run

Then:

Baseline
vs
SWARMOS

Show real measurements.

Use clear units.

Do not bury the most important comparison below charts.

=====

REPLAY SCREEN

=====

Primary element:
time navigation + historical warehouse state.

Secondary:
event rail

Context:
robot/task/incident inspector

This screen should support investigation.

=====

ANALYTICS SCREEN

=====

Do not make this a chart museum.

Group insights by question:

FLEET PERFORMANCE

TRAFFIC / BOTTLENECKS
TASK PERFORMANCE
COORDINATION
RECOVERY

SCALING

Every chart must have purpose.

SYSTEM HEALTH

Show only actual system information.

Possible:

realtime connection
simulation state
frontend data freshness
event throughput
latency
system errors

Never fake health.

PROFESSIONAL UX PRINCIPLE

Every major workflow should follow:

MONITOR

→

UNDERSTAND

→

DECIDE

→

ACT

→

VERIFY

Example:

Incident

→ investigate

→ understand

→ act

→ observe result

→ verify recovery

This should be the backbone of the experience.

CORE QUESTIONS THE UI MUST ANSWER

At any important moment, the operator should be able to answer:

WHAT is happening?

WHERE?

WHICH robot?

WHICH task?

HOW serious?

WHY?

WHAT is SWARMOS doing?

WHAT can I do?

WHAT happened afterward?

COGNITIVE LOAD

Do not solve complexity by hiding useful information.

Use:

- hierarchy
- grouping
- filtering
- progressive disclosure

- contextual inspectors
- focus mode
- exception prioritization

Show the right amount of information at the right time.

=====

SITUATIONAL AWARENESS

=====

Design for:

PERCEPTION

→

COMPREHENSION

→

PROJECTION

The interface should help the operator understand:

what is happening now

why it is happening

what is likely to happen next

Only show predictions where actual functionality supports them.

=====

ALERT FATIGUE

=====

Avoid:

- constant alerts
- repeated notifications
- flashing warnings
- equal severity for every issue

Prioritize.

Group.

Provide actionable context.

=====

PERFORMANCE REQUIREMENTS

=====

Primary target:

50 continuously moving AMRs.

Stress:

100+ where practical.

Measure:

- FPS
- CPU
- memory
- render cost
- React rerenders
- interaction latency
- map responsiveness
- list performance
- chart performance

Find and fix:

- unnecessary rerenders
- memory leaks
- expensive event handlers
- excessive DOM
- unnecessary state updates

Do not blindly optimize.

Profile.

=====

REALTIME UX

=====

Where the existing backend provides realtime data:

consume it correctly.

Handle:

- initial snapshot
- incremental updates
- reconnect
- stale state
- duplicate events
- out-of-order events where relevant

The interface should visibly communicate data freshness.

STATE MODEL

Keep distinct:

SERVER / APPLICATION STATE
REALTIME STATE
UI STATE
REPLAY STATE
VISUAL RENDER STATE

High-frequency robot motion should not force slow-changing interface elements to rerender.

NO TECHNOLOGY SOUP

Do not add libraries simply because they are trendy.

Use the existing stack where reasonable.

Possible technologies include:

React
TypeScript
Tailwind
shadcn/Radix
Motion
Canvas
WebGL
PixiJS

Three.js
React Three Fiber
Recharts
other justified tools

Select only what is technically justified.

TESTING

Test every significant workflow.

NAVIGATION

- every page opens
- no dead destinations

COMMAND PALETTE

- every command works

SEARCH

- results are correct
- selection leads to correct context

MAP

- pan
- zoom
- selection
- layers
- focus

ROBOT

- movement
- selection
- inspector
- task context

TASK

- create
- filter
- sort
- inspect
- reassign
- cancel
- reprioritize where supported

INCIDENT

- open
- locate
- action
- resolution

SIMULATION

- preset
- start
- pause
- resume
- speed
- reset
- scenario changes

REPLAY

- play
- pause
- seek
- event jump
- historical state

BENCHMARK

- actual data
- correct calculations
- actual charts

ANALYTICS

- correct filters
- correct data

=====

SIMULATION INVARIANTS

=====

Where applicable, verify:

1. robots cannot occupy impossible states
2. robot movement corresponds to simulation state
3. task state transitions are consistent
4. incident state is consistent with events
5. replay reconstructs historical state
6. benchmark metrics are derived from runs
7. simulation reset actually resets
8. scenario parameters actually affect simulation
9. failure affects the relevant robot/task
10. rerouting changes the actual route representation

=====

DATA VISUALIZATION TEST

=====

For every chart:

verify:

- source
- metric definition
- units
- timestamps
- scenario
- run
- filtering

Do not let a chart display stale data after a scenario changes.

=====

VISUAL QA

=====

After implementation, inspect every screen critically.

Ask:

Does this look like a real robotics product?

Does it look intentionally designed?
Is the hierarchy clear?
Is the map the hero where appropriate?
Do the robots look like actual AMRs?
Can I see them moving?
Can I understand what they are doing?
Can I understand conflict?
Can I understand rerouting?
Can I understand failure?
Can I see task reassignment?
Do the metrics update from real data?
Do the graphs represent real data?
Is anything fake?
Is anything decorative without purpose?
Is anything too flashy?
Is anything too dense?
Is anything too small?
Is anything confusing?
Fix every important problem.

=====

HARSH SIH JUDGE REVIEW

=====

Review the final product as if you are a highly demanding judge.
Try to reject it.

Ask:

"Is this just another dashboard?"
"Are the robots actually working?"
"Is this only animation?"
"Are the charts fake?"
"What exactly is the distributed intelligence?"
"Can I see a real coordination decision?"
"Can I see why a robot yielded?"
"Can I see a real conflict?"
"Can I see recovery after a failure?"
"Can I measure the improvement?"
"Can I understand the warehouse?"
"Is the visual design generic?"
"Is the simulation toy-like?"
"Could another team reproduce this by changing colors?"
"Does the system have real depth or only visual polish?"

If any answer exposes a weakness:
fix it.

=====

IMPORTANT: THE WOW MUST COME FROM SYSTEM BEHAVIOR

=====

The most impressive moment should NOT be:

a glowing animation
a futuristic background
a flashy chart
a 3D camera transition

The strongest moment should be:

50 AMRs are actively working.

An operational problem appears.

SWARMOS identifies it.

The system explains it.

A distributed decision is made.

The robots visibly respond.

The routes change.

A failure occurs.

The task is reassigned.

The fleet recovers.

The actual measured results appear.

That is the WOW.

=====

IMPLEMENTATION PRIORITY

=====

P0 – MUST BE EXCELLENT

- Live Warehouse
- realistic warehouse environment
- 50 AMRs
- actual robot movement
- tasks
- routes
- robot selection
- contextual robot inspector
- simulation controls
- conflict visualization
- rerouting visualization
- incident center
- real data integrity
- realtime state
- core navigation
- functional controls
- no fake UI

P1 – HIGH VALUE

- mission composer
- decision inspector
- deadlock visualization
- congestion heatmap
- replay
- benchmarks
- analytics
- bottleneck analysis
- command palette
- advanced scenario builder
- edge/decision view

P2 – ADD WHERE ACTUALLY SUPPORTED

- advanced 3D mode
- additional analytical views
- role-specific customization
- more export functionality
- advanced comparison capabilities

Do not sacrifice P0 reliability for P2 decoration.

=====

FINAL IMPLEMENTATION RULE

Do not implement the product as disconnected pages.

All entities must remain coherent.

The same:

ROBOT

should remain the same object across:

- Live Map
- Fleet
- Task Inspector
- Incident Inspector
- Replay
- Analytics where applicable

The same:

TASK

must preserve identity and state across the application.

The same:

INCIDENT

must preserve context.

The same:

SIMULATION RUN

must feed:

- live visualization
- events
- replay
- metrics
- benchmark

This must feel like one system.

FINAL ACCEPTANCE CRITERIA

Do NOT consider the work complete until all applicable conditions are true:

- [] Existing useful functionality has been audited.
- [] Useful working functionality has been preserved.
- [] Useful broken functionality has been repaired.
- [] The UI has been substantially elevated.
- [] The warehouse looks credible.
- [] The warehouse is the visual environment.
- [] There is no generic AI background.
- [] There is no cyberpunk aesthetic.
- [] There is no excessive neon/glow.
- [] There is no excessive glassmorphism.
- [] The robots do not look like buttons.
- [] 50 AMRs can operate visibly.
- [] Robots actually move.
- [] Robots have meaningful tasks.
- [] Movement is smooth.
- [] Routes are understandable.

[] Rerouting is visible.

[] Conflicts are visible.

[] Deadlocks are understandable where supported.

[] Blockages affect the fleet.

[] Failures affect the fleet.

[] Task reassignment is visible where supported.

[] Recovery is visible.

[] Mission/task management works.

[] Robot inspector works.

[] Task inspector works.

[] Incident inspector works.

[] Decision inspector works where data exists.

[] Simulation controls work.

[] Scenario presets work.

[] Scenario configuration affects the actual simulation.

[] Replay works.

[] Benchmarks use real measured data.

[] Charts use real data.

[] No fake metrics exist.

[] No fake graphs exist.

[] No fake robot activity exists.

[] No dead buttons exist.

[] Navigation works.

[] Search works.

[] Command palette works.

[] Layer toggles work.

[] Error states exist.

[] Loading states exist.

[] Empty states exist.

[] Stale/disconnected state is communicated.

[] Keyboard navigation works.

[] Critical states do not rely on color alone.

[] Reduced motion is supported.

[] Performance has been tested with 50 robots.

[] Stress behavior has been tested where practical.

[] No obvious memory leaks exist.

[] No obvious rendering bottlenecks remain.

[] Typography is readable.

[] Visual hierarchy is strong.

[] The application feels like one coherent product.

=====

FINAL SELF-CRITIQUE LOOP

=====

Do not stop after the first implementation.

Perform this cycle:

AUDIT
→ IMPLEMENT
→ RUN
→ TEST
→ INSPECT
→ IDENTIFY WEAKNESSES
→ FIX
→ PERFORMANCE TEST
→ ACCESSIBILITY TEST
→ DATA INTEGRITY TEST
→ VISUAL QA
→ HARSH JUDGE REVIEW
→ POLISH
→ TEST AGAIN

Do not accept:

"looks good."

Require:

"works correctly, communicates clearly, performs well, and survives critical review."

=====

FINAL INSTRUCTION

=====

Now inspect the existing SWARMOS project.

Do not immediately rewrite everything.

First understand the current implementation.

Then execute the transformation.

Preserve useful functionality.

Repair broken functionality.

Restructure weak UX.

Implement missing high-value UX.

Build the digital twin properly.

Make the 50 AMRs actually work.

Make the simulation feel alive.

Make conflicts understandable.

Make decisions explainable.

Make failures meaningful.

Make recovery visible.

Make analytics truthful.

Make graphs real.

Make interactions functional.

Make the interface premium.

Make the product calm, precise and trustworthy.

Do NOT create:

"a prettier student dashboard."

Create:

"A professional robotics fleet command-and-control experience."

The final experience must communicate this story without requiring the judge to imagine anything:

50 AMRs working
→ real tasks progressing
→ traffic developing
→ congestion
→ conflict

- distributed decision
- action
- resolution
- blockage
- rerouting
- robot failure
- task reassignment
- recovery
- actual measurements
- benchmark
- real analytics

The warehouse should feel alive.

The interface should make the invisible intelligence visible.

The data should be trustworthy.

The interaction should be deliberate.

The motion should have meaning.

The visual design should be distinctive without gimmicks.

The product should be impressive because it is GOOD – not because it is flashy.

=====
END OF MASTER PROMPT
=====